Aakaar Frontend — Container & Deployment

Vite + React + TypeScript SPA (**aakar-web/**) packaged as a two-stage build: Node 20 produces the static bundle, then nginx 1.27-alpine serves it and reverse proxies **/api** to the backend.

1. Image at a glance

Field	Value
Builder	node:20-alpine
Runtime	nginx:1.27-alpine
Working dir	/usr/share/nginx/html
Exposed port	80/tcp
Build arg	VITE_API_BASE (default /api, baked into bundle)
Runtime env	AAKAR_BACKEND_URL (default http://aakar-api:8000)
Healthcheck	GET http://127.0.0.1/healthz (every 30s)
Final image size	≈35 MiB compressed

2. Dockerfile

Path: **aakar-web/Dockerfile**.

```
# syntax=docker/dockerfile:1.7
# ---- Aakaar frontend image ------------------------------------------------
# Stage 1: build the Vite bundle. Stage 2: serve static assets from nginx.

# ----- builder ---------------------------------------------------------------
FROM node:20-alpine AS builder

WORKDIR /app

# VITE_API_BASE is baked into the bundle at build time. Leave empty so the
# bundle calls /api and let nginx (next stage) reverse-proxy to the backend.
ARG VITE_API_BASE=/api
ENV VITE_API_BASE=$VITE_API_BASE

COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund

COPY tsconfig*.json vite.config.ts postcss.config.js tailwind.config.ts index.html ./
COPY public ./public
COPY src ./src
RUN npm run build

# ----- runtime ---------------------------------------------------------------
FROM nginx:1.27-alpine AS runtime

# AAKAR_BACKEND_URL is read by the entrypoint to template nginx.conf.
ENV AAKAR_BACKEND_URL=http://aakar-api:8000

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/templates/default.conf.template

# nginx:alpine already ships docker-entrypoint.sh that envsubst's any file
# under /etc/nginx/templates into /etc/nginx/conf.d/.

EXPOSE 80

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
```

```
  CMD wget -qO- http://127.0.0.1/healthz >/dev/null || exit 1
```

3. nginx config

Path: **aakar-web/nginx.conf**. Templated via **${AAKAR_BACKEND_URL}** at container start by the upstream nginx image's envsubst entrypoint. SPA fallback and gzip are configured; immutable assets get a 1y cache.

```
server {
    listen       80;
    server_name  _;

    root   /usr/share/nginx/html;
    index  index.html;

    # gzip text assets
    gzip              on;
    gzip_comp_level   5;
    gzip_min_length   256;
    gzip_proxied      any;
    gzip_types        text/plain text/css application/javascript application/json
                      application/xml application/wasm image/svg+xml;

    # immutable hashed assets get a long cache; html stays fresh
    location ~* \.(?:js|css|woff2?|ttf|eot|svg|png|jpg|jpeg|gif|webp|ico)$ {
        try_files $uri =404;
        access_log off;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location = /healthz {
        access_log off;
        return 200 "ok\n";
    }

    # Reverse proxy /api/* to the backend. Strip /api so /api/auth/login
    # becomes /auth/login at the FastAPI side (matches the dev proxy).
    location /api/ {
        proxy_pass         ${AAKAR_BACKEND_URL}/;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_set_header   Upgrade           $http_upgrade;
        proxy_set_header   Connection        "upgrade";
        proxy_read_timeout 300s;
        proxy_send_timeout 300s;
        client_max_body_size 50m;
    }

    # SPA fallback — every unknown path serves index.html so React Router
    # can resolve client-side routes.
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

4. Build & run

Build with the default same-origin API base:

```
docker build -t aakaar/web:latest -f aakar-web/Dockerfile aakar-web
```

Override **VITE_API_BASE** if the API is on a different origin and you do not want the bundled nginx proxy:

```
docker build \
```

```
  --build-arg VITE_API_BASE=https://api.aakaar.example \
  -t aakaar/web:latest -f aakar-web/Dockerfile aakar-web
```

Run pointing at an upstream API:

```
docker run --rm -p 8080:80 \
  -e AAKAR_BACKEND_URL=http://aakar-api.internal:8000 \
  aakaar/web:latest
```

5. Caching & invalidation

- Vite emits hashed filenames (e.g. **index-9f2a.js**); they are served with **Cache-Control: public, immutable, max-age=31536000**.
- **index.html** is served without immutable caching, so a redeploy flips clients onto the new bundle on next navigation. No CDN invalidation step is required if the CDN respects upstream Cache-Control.
- Behind a CDN (CloudFront/Cloudflare), set the origin response policy to honor Cache-Control headers; do not override **index.html** with a long TTL.

6. Routing

All non-asset paths fall back to **/index.html** so React Router resolves them client-side. **/api/*** is reverse-proxied to the backend with **/api** stripped, matching the dev server proxy in **vite.config.ts**.

7. Resource sizing

Profile	Recommended limits
Single replica	0.1 vCPU, 64 MiB
Per replica beyond 100 RPS	0.25 vCPU, 128 MiB

Static serving is cheap; scale horizontally only if the load balancer measures elevated p95 latency at the nginx layer (rare). The backend will saturate first.

8. Security headers (recommended add-ons)

If the deployment terminates TLS at this nginx (rather than at an upstream LB), extend **nginx.conf** with the headers below. They are intentionally not in the shipped config so they do not double-stack with a fronting CDN.

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; \
  script-src 'self'; style-src 'self' 'unsafe-inline'; \
  img-src 'self' data: blob:; connect-src 'self' https://api.aakaar.example;" always;
```